DAY 17

Django REST Framework Fundamentals

A compact guide to the sample API project

PROJECT	WHAT IT RETURNS
Hello World API	A JSON greeting from a GET endpoint.
Student API	Three sample student records serialized as JSON.

Start the project

Open a terminal in the **day17_django_api** folder. Install Django and Django REST Framework, apply the starter database migrations, then launch Django's development server:

```
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Open or send GET requests to these URLs:

URL	EXPECTED JSON
/api/hello/	{"message":"Hello, World!"}
/api/students/	A JSON array with three student objects.

The DRF browsable API is also available in a browser. The endpoints in this exercise accept GET requests.

How the project is organized

Django separates project configuration from application features. This example keeps the API routes and student response logic in a dedicated app.

```
day17_django_api/
|-- manage.py                # Django command-line entry point
|-- requirements.txt         # Django and DRF dependencies
|-- config/
|   |-- settings.py          # Installed apps, middleware, database
|   |-- urls.py              # Project-level URL routing
|   |-- asgi.py / wsgi.py    # Server entry points
|-- students/
    |-- urls.py              # API route paths
    |-- views.py             # GET views and sample data
    |-- serializers.py       # Student response shape
```

Request and response path

1 URL	2 API view	3 Serializer	4 Response
The project includes /api/ routes and the app matches hello/ or students/.	The view handles the GET request and selects the greeting or sample records.	StudentSerializer shapes each record into fields with declared types.	DRF Response returns JSON to the browser or API client.

Concepts practiced

Project and app	The project configures Django; the students app groups API functionality.
URL routing	config/urls.py mounts the app under /api/, and students/urls.py maps route names to views.
API views	@api_view(["GET"]) restricts these function views to GET requests.
Serializer	StudentSerializer declares the fields and types for outgoing student data.
HTTP response	Response converts Python dictionaries and lists into a JSON response.

Key implementation

The sample data is kept in memory for this fundamentals exercise. The API exposes it through a serializer and read-only GET views.

Serializer

```
class StudentSerializer(serializers.Serializer):
    id = serializers.IntegerField()
    name = serializers.CharField()
    age = serializers.IntegerField()
    course = serializers.CharField()
```

API views

```
@api_view(["GET"])
def hello_world(request):
    return Response({"message": "Hello, World!"})

@api_view(["GET"])
def student_list(request):
    serializer = StudentSerializer(SAMPLE_STUDENTS, many=True)
    return Response(serializer.data)
```

Try it in a browser or Postman

Start runserver and visit **http://127.0.0.1:8000/api/hello/**. You should see a JSON object with a message field. Then visit **http://127.0.0.1:8000/api/students/** and confirm that the response is a list of three records, each containing id, name, age, and course. In Postman, choose GET and use the same URLs.

Expected student response shape

```
[
  {"id": 1, "name": "Aarav Sharma", "age": 20, "course": "Computer Science"},
  {"id": 2, "name": "Maya Patel", "age": 21, "course": "Information Technology"},
  {"id": 3, "name": "Rohan Das", "age": 19, "course": "Data Science"}
]
```

For practice, add another student, create a detail route by ID, then explore POST and validation in a follow-up exercise.